☁ Cloud App - [LenteraPustaka]

Aplikasi **LenteraPustaka** adalah sebuah platform berbasis web yang dirancang untuk mendigitalisasi proses pengelolaan katalog dan transaksi peminjaman buku di perpustakaan. Aplikasi ini bertujuan untuk memberikan kemudahan akses informasi bagi pengunjung perpustakaan, memfasilitasi sirkulasi peminjaman, sekaligus menyederhanakan tugas administratif pustakawan secara terpusat.

Sistem ini dirancang untuk tiga kelompok demografi pengguna dengan kebutuhan yang berbeda:

1. **Pengunjung Publik (Guest):**
   mahasiswa, pelajar, atau masyarakat umum yang ingin menelusuri katalog atau cek ketersediaan buku sebelum memutuskan untuk mendaftar
2. **Anggota Terdaftar (Member):**
   pengguna aktif perpustakaan yang membutuhkan fasilitas untuk meminjam buku, mengecek batas waktu pengembalian, dan melihat riwayat.
3. **Pustakawan (Admin):**
   Petugas perpustakaan yang membutuhkan alat bantu efisien berupa dashboard statistik terstruktur dan mengelola inventaris, menyetujui peminjaman, dan mengelola member.

Aplikasi ini dikembangkan untuk memberikan solusi atas beberapa kendala dalam pengelolaan perpustakaan:

1. **Pencarian yang inefisien:**
   Menyelesaikan masalah pengunjung yang kesulitan menemukan buku di rak fisik dengan menyediakan fitur *searchbar* digital yang cepat dan akurat.
2. **Risiko Kehilangan Data Transaksi:**
   Menggantikan pencatatan peminjaman dengan menjadi basis data digital yang terstruktur.
3. **Keterbatasan Akses Informasi:**
   Mengatasi masalah pengunjung yang harus datang langsung hanya untuk mengecek apakah sebuah buku sedang dipinjam orang lain atau tersedia dengan menampilkan ketersediaan stok secara real-time

👥 Tim

Nama	NIM	Peran
Maulana Malik Ibrahim	10231051	Lead Backend
Micka Mayulia Utama	10231053	Lead Frontend
Khanza Nabila Tsabita	10231049	Lead DevOps
Muhammad Aqila Ardhi	10231057	Lead QA & Docs

🛠 Tech Stack

Teknologi	Fungsi

Teknologi	Fungsi
FastAPI	Backend REST API
React	Frontend SPA
PostgreSQL	Database
Docker	Containerization
GitHub Actions	CI/CD
Railway/Render	Cloud Deployment

🏗️ Architecture

```
[React Frontend] <--HTTP--> [FastAPI Backend] <--SQL--> [PostgreSQL]
       |                           |
  Vite + JSX               REST API Endpoints
(Port 3000/5173)              (Port 8000)
```

🤖 Getting Started

Kami merekomendasikan penggunaan **Docker Compose** agar seluruh sistem dapat berjalan serentak secara otomatis. Namun, kami juga menyediakan panduan *Quick Start* (Native) untuk kebutuhan *development* atau pengujian manual per modul.

Kami telah menyediakan panduan langkah-demi-langkah yang lengkap untuk menjalankan proyek ini di berbagai perangkat.

🐳 Panduan Menjalankan Project (Docker Compose)

Proyek ini sudah terkonfigurasi penuh menggunakan Docker Compose untuk memudahkan proses deployment lokal.

1. Jalankan Seluruh Sistem

Buka terminal di root folder proyek dan jalankan perintah:

```
docker compose up -d
```

(Perintah ini akan secara otomatis membuat network lentera_net, menyiapkan volume lentera_data, dan menyalakan Database, Backend, serta Frontend secara bersamaan).

2. Cek Status Aplikasi

Pastikan ketiga services sudah berstatus Up (healthy):

```
docker compose ps
```

3. Akses Aplikasi

- Frontend (UI): http://localhost:3000
- Backend API: http://localhost:8000
- API Documentation (Swagger): http://localhost:8000/docs

4. Mematikan Sistem

Untuk mematikan sistem tanpa menghilangkan data database:

```
docker compose down
```

Quick Start Instalasi Native (Untuk Development)

Prasyarat

- Python 3.10+
- Node.js 18+
- Git

Automated Setup: `./setup.sh`

- Backend

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

- Frontend

```
cd frontend
npm install
npm install recharts
npm run dev
```

Access

- API: http://localhost:8000
- API Docs: http://localhost:8000/docs
- Frontend: http://localhost:5173

📅 Roadmap

Minggu	Target	Status
1	Setup & Hello World	☑
2	REST API + Database	☑
3	React Frontend	☑
4	Full-Stack Integration	☑
5-7	Docker & Compose	☑
8	UTS Demo	☐
9-11	CI/CD Pipeline	☐
12-14	Microservices	☐
15-16	Final & UAS	☐

📁 Project Structure

```
CC-KELOMPOK-A-HEXACORE/
├── backend/
│   ├── static/
│   ├── Dockerfile           ← BARU
│   ├── .dockerignore        ← BARU
│   ├── crud.py              ← Updated (Fix Lost Buku Status)
│   ├── database.py
│   ├── main.py              ← Updated (Fix Lost Buku Status)
│   ├── models.py
│   ├── requirements.txt
│   ├── schemas.py
│   ├── .env                 ← Updated (host.docker.internal)
│   └── .env.example         ← Updated
├── docs/
│   ├── test/
│   ├── api-documentation.md
│   ├── api-test-results.md
│   ├── auth-test-results.md
│   ├── docker-cheatsheet.md  ← BARU
│   ├── image-comparison.md  ← BARU
│   ├── env-setup.md
│   ├── schemadatabase.md
│   ├── member-aqila.md
│   ├── member-Khanza_Nabila_Tsabita.md
│   ├── member-Maulana_Malik_Ibrahim.md
│   ├── member-Micka_Mayulia_Utama.md
│   ├── setup-guide.md
│   └── ui-test-result.md
├── frontend/
```

```
│   ├── frontend/
│   ├── node_modules/
│   ├── public/
│   └── src/                ← Update
│       ├── App.jsx                 ← Update
│       ├── App.css                 ← Update
│       ├── main.jsx                ← Update
│       └── components/
│           ├── Header.jsx          ← Update
│           ├── SearchBar.jsx       ← Update
│           ├── ItemForm.jsx        ← Update
│           ├── ItemList.jsx        ← Update
│           └── ItemCard.jsx        ← Update
├── .gitignore
├── setup.sh
└── README.md                       ← Update
```

🗀 Tabel ERD

```
+-------------------+              +-----------------------+
|       USERS       |              |      TRANSACTIONS     |
+-------------------+              +-----------------------+
| user_id (PK)      | 1          N | transaction_id (PK)   |
| email (UK)        |--------------| user_id (FK)          |
| password_hash     | (Melakukan)  | book_id (FK)          |
| full_name         |              | borrow_date           |
| role              |              | due_date              |
| created_at        |              | return_date           |
+-------------------+              | status                |
                                   +-----------+-----------+
                                         | 1         | N
                                         |           |
+-------------------+     (Menghasilkan) |           |
|    CATEGORIES     |                    |           |
+-------------------+                    | 1         |
| category_id (PK)  |              +-------------+   |
| name (UK)         |              |   FINES     |   |
| description       |              +-----------  +   |
+---------+---------+              | fine_id(PK) |   |
          |                        | trans_id(FK)|   |
          | 1                      | amount      |   |
          | (Memiliki)             | status      |   |
          |                        | proof_url   |   |
          | N                      | rej_note    |   |
+---------+---------+              +-----------+     |
|       BOOKS       | 1                               |
+-------------------+---------------------------------+
| book_id (PK)      |
| category_id (FK)  |
| isbn (UK)         |
```

```
| title              |
| author             |
| publisher          |
| publication_year   | 1     N +-----------------+ N     1 +------------------+
| synopsis           |---------|   BOOK_GENRES   |---------|      GENRES      |
| total_stock        |         +-----------------+         +------------------+
| available_stock    |         | book_id (PK,FK) |         | genre_id (PK)    |
| cover_image_url    |         | genre_id(PK,FK) |         | name (UK)        |
+--------------------+         +-----------------+         | description      |
                                                           +------------------+
```

Berikut adalah detail arsitektur *database* PostgreSQL yang digunakan oleh aplikasi LenteraPustaka.

- Schema Database

Dokumentasi Teknis & Laporan Pengujian

Modul 2: Backend REST API (FastAPI)

- Hasil Pengujian API Terintegrasi via Swagger
- Ringkasan Spesifikasi & Parameter API

Modul 3: Frontend Development (React UI)

- Hasil Pengujian Fungsionalitas Antarmuka (UI)

Modul 4: Integrasi Full-Stack & Autentikasi

- Hasil Pengujian End-to-End (Alur Otentikasi & Otorisasi)
- Panduan Instalasi & Konfigurasi Environment Lokal
- Dokumentasi Lengkap Endpoints API

Modul 5 & 6: Docker Containerization & Orchestration

- **Diagram Arsitektur Multi-Kontainer Docker**
- Analisis & Perbandingan Ukuran Base Image Docker
- Cheat Sheet: Daftar Perintah Esensial Docker

Persiapan UTS & Arsitektur Utama

- **Naskah Demo UTS Kelompok Hexacore**
- **Detail Skema Database & ERD Terkini**